IES San Mamede

Ciclo Formativo de Grado Superior

Desarrollo de Aplicaciones Multiplataforma (DAM)

NetManagement

Sistema de Monitorización y Control de Equipos en Aulas Informáticas

Manual de Instalación

Autor: Enooc Domínguez Quiroga

Centro: IES San Mamede

Curso: 2025 / 2026

Fecha: Junio de 2026

Versión 1.0 — Junio 2026

Índice

1. Introducción **2**

2. Requisitos previos **2**
2.1. Requisitos del servidor 2
2.2. Requisitos del agente cliente 2
2.3. Requisitos de los equipos de red 3

3. Arquitectura del despliegue **3**
3.1. Redes Docker 3

4. Instalación del servidor **4**
4.1. Paso 1: Clonar el repositorio 4
4.2. Paso 2: Configurar las variables de entorno 4
4.3. Paso 3: Despliegue automático 5
4.4. Paso 4: Despliegue manual (alternativa) 5
4.5. Paso 5: Verificar el despliegue 6

5. Configuración de los equipos de red MikroTik **6**
5.1. Habilitar la API 6
5.2. Crear usuario de API 6
5.3. Certificados TLS para la API 7
5.4. Registrar el equipo de red en la aplicación 7

6. Instalación del agente cliente **7**
6.1. Configuración previa 7
6.2. Instalación en Windows 8
6.3. Instalación en Linux 9
6.4. Verificación 9

7. Configuración inicial de la aplicación **10**

8. Comandos de administración **11**

9. Solución de problemas **12**

10. Estructura de ficheros del despliegue **12**

1. Introducción

Este documento describe los pasos necesarios para desplegar el sistema **NetManagement** en un entorno de producción. El sistema se compone de tres elementos principales:

1. **Servidor (backend + frontend):** Aplicación web desplegada con Docker Compose que incluye el backend Django, la base de datos PostgreSQL, Redis, el frontend React y el proxy inverso Caddy.
2. **Agente cliente:** Programa Python que se instala en cada equipo del aula y se comunica con el servidor vía WebSocket.
3. **Equipos de red MikroTik:** Switches/routers gestionados por el backend mediante su API para aplicar reglas de aislamiento de red.

2. Requisitos previos

2.1. Requisitos del servidor

Herramienta	Versión mínima	Enlace
Docker	24+	https://docs.docker.com/engine/install/
Docker Compose	v2+	Incluido con Docker Desktop / plugin CLI
Bun	1.0+	https://bun.sh
Git	2.x	https://git-scm.com

Cuadro 1: Requisitos de software del servidor.

- **Sistema operativo:** Cualquier sistema compatible con Docker (Linux recomendado, Windows con WSL2, macOS).
- **Puertos:** El servidor debe tener los puertos `80` y `443` accesibles desde la red local (y desde Internet si se desea TLS automático con Let's Encrypt).
- **Conectividad de red:** El servidor debe poder alcanzar las direcciones IP de los equipos de red MikroTik del centro.

Nota: Si se prefiere `npm` o `yarn` en lugar de Bun, basta con sustituir `bun install` y `bun run build` por los comandos equivalentes.

2.2. Requisitos del agente cliente

Herramienta	Versión mínima	Observaciones
Python	3.10+	Incluido en la mayoría de distribuciones Linux
pip	21+	Se instala automáticamente con el instalador

Cuadro 2: Requisitos del agente cliente.

- **Sistemas operativos soportados:** Windows 10/11, Linux (X11/Xorg).
- **Limitación en Linux:** La captura de pantalla requiere el servidor gráfico **X11/Xorg**. Wayland no está soportado por la librería `mss`.

2.3. Requisitos de los equipos de red

- Router/switch **MikroTik** con RouterOS 6.x o 7.x.
- **API habilitada** en `/ip/service` (puerto 8728 sin TLS o 8729 con TLS).
- Usuario con permisos de lectura y escritura sobre `/ip/firewall/filter`, `/interface/bridge/fil` `/interface/bridge/port` y `/ip/arp`.

3. Arquitectura del despliegue

El despliegue en producción utiliza Docker Compose con la siguiente arquitectura:

Arquitectura de servicios

```
Internet --80/443--> Caddy (proxy inverso + TLS automatico)
                       |-- /api/*    --> backend:8000  (Django
                          REST)
                       |-- /ws/*     --> backend:8000  (Daphne
                          WebSocket)
                       |-- /admin/*  --> backend:8000  (Django
                          Admin)
                       |-- /static/* --> ficheros estaticos
                          Django
                       '-- /*        --> React SPA (Vite build)

                    Backend --> PostgreSQL 16
                            --> Redis 7 (Channel Layers +
                               Heartbeat)
                            --> Routers MikroTik (API TLS puerto
                               8729)
```

3.1. Redes Docker

Red	Tipo	Propósito
`backend_net`	bridge (no interna)	Comunicación entre servicios base (DB, Redis, Backend). No debe ser `internal: true` porque el backend necesita alcanzar los MikroTik de la red local.
`frontend_net`	bridge	Conecta Caddy con el backend y permite la exposición de puertos al exterior.

Cuadro 3: Redes Docker del despliegue.

Solo **Caddy** expone puertos al exterior (80 y 443). Los demás servicios no son accesibles directamente desde fuera de Docker.

4. Instalación del servidor

4.1. Paso 1: Clonar el repositorio

```
git clone https://github.com/enoocdev/proyecto_dam.git
cd proyecto_dam
```

4.2. Paso 2: Configurar las variables de entorno

```
cd produccion
cp .env.example .env
```

Editar el fichero `.env` con los valores adecuados para el entorno:

Ejemplo de .env

```
# --- PostgreSQL ---
POSTGRES_DB=monitor_db
POSTGRES_USER=postgres
POSTGRES_PASSWORD=tu_password_segura

# --- Django ---
SECRET_KEY=clave-secreta-generada
DEBUG=False
ALLOWED_HOSTS=tu-dominio.com,www.tu-dominio.com,localhost
CORS_ALLOWED_ORIGINS=https://tu-dominio.com

# --- Superusuario inicial ---
DJANGO_SUPERUSER_USERNAME=admin
DJANGO_SUPERUSER_PASSWORD=password_segura
DJANGO_SUPERUSER_EMAIL=admin@tu-dominio.com

# --- Caddy (dominio para TLS automatico) ---
# Usar dominio real para Let's Encrypt, o :80 para pruebas
DOMAIN=tu-dominio.com
```

Generar la clave secreta de Django:

```
cd ../backend
python generar_clave.py
```

Copiar el resultado en la variable `SECRET_KEY` del fichero `.env`.

4.3. Paso 3: Despliegue automático

El repositorio incluye un script que automatiza todo el proceso:
Linux / macOS:

```
cd produccion
chmod +x deploy.sh
./deploy.sh
```

Windows:

```
cd produccion
deploy.bat
```

El script realiza automáticamente las siguientes operaciones:

1. Instala las dependencias del frontend (`bun install`).
2. Compila el frontend para producción (`bun run build`).
3. Construye la imagen Docker del backend.
4. Levanta todos los contenedores con Docker Compose.
5. Ejecuta las migraciones de base de datos.
6. Recopila los ficheros estáticos de Django.
7. Crea el superusuario con las credenciales del `.env`.
8. Extrae los certificados TLS de Caddy para la configuración de MikroTik.

4.4. Paso 4: Despliegue manual (alternativa)

Si se prefiere ejecutar cada paso por separado:

```
# 1. Compilar el frontend
cd frontend
bun install
bun run build
cd ..

# 2. Construir y levantar contenedores
cd produccion
docker compose -f docker-compose.prod.yml build
docker compose -f docker-compose.prod.yml up -d --wait

# 3. Ejecutar migraciones
docker compose -f docker-compose.prod.yml exec backend \
    python manage.py migrate --noinput

# 4. Recopilar archivos estaticos
docker compose -f docker-compose.prod.yml exec backend \
```

```
    python manage.py collectstatic --noinput

# 5. Crear superusuario
docker compose -f docker-compose.prod.yml exec backend \
    python manage.py createsuperuser --noinput
```

4.5. Paso 5: Verificar el despliegue

```
# Ver estado de los servicios
docker compose -f docker-compose.prod.yml ps

# Ver logs en tiempo real
docker compose -f docker-compose.prod.yml logs -f
```

Acceder a la aplicación web desde el navegador en `https://tu-dominio.com` (o `http://localhost` si se configuró `DOMAIN=:80`).

5. Configuración de los equipos de red MikroTik

Para que el backend pueda aplicar reglas de aislamiento de red, es necesario configurar cada equipo MikroTik del centro.

5.1. Habilitar la API

Acceder a la consola del router (por WinBox, SSH o terminal) y ejecutar:

RouterOS CLI

```
# Habilitar API sin TLS (puerto 8728, solo para pruebas)
/ip service enable api

# Habilitar API con TLS (puerto 8729, recomendado en produccion)
/ip service enable api-ssl
```

5.2. Crear usuario de API

Se recomienda crear un usuario específico para el backend con los permisos necesarios:

RouterOS CLI

```
/user group add name=api-monitor \
    policy=read,write,api,test

/user add name=netmanagement group=api-monitor \
    password=password_segura
```

5.3. Certificados TLS para la API

Si se utiliza el puerto con TLS (8729), el script de despliegue extrae automáticamente los certificados de Caddy en la carpeta `produccion/mikrotik_certs/`:

- `caddy_root.crt` — Certificado raíz de la CA.
- `server.crt` — Certificado del servidor.
- `server.key` — Clave privada del servidor.

Subir estos ficheros al MikroTik e importarlos:

RouterOS CLI

```
/certificate import file-name=caddy_root.crt passphrase=""
/certificate import file-name=server.crt passphrase=""
/certificate import file-name=server.key passphrase=""
```

Asignar el certificado al servicio API-SSL:

RouterOS CLI

```
/ip service set api-ssl certificate=server.crt_0
```

5.4. Registrar el equipo de red en la aplicación

Una vez configurado el MikroTik, acceder al panel de administración web (`/admin` o desde la interfaz) y crear un nuevo **Equipo de Red** con:

- **Nombre:** Identificador descriptivo (ej.: *Switch Aula 1*).
- **Dirección IP:** IP del MikroTik accesible desde el servidor Docker.
- **Usuario:** El usuario de API creado anteriormente.
- **Contraseña:** La contraseña del usuario de API.
- **Puerto API:** `8728` (sin TLS) o `8729` (con TLS).

6. Instalación del agente cliente

El agente debe instalarse en cada equipo del aula que se desee monitorizar.

6.1. Configuración previa

Antes de instalar, editar el fichero `client/client_config.json` (o usar variables de entorno) para establecer la URL del servidor:

client_config.json

```
{
    "WS_URL": "wss://tu-dominio.com/ws/client/",
    "HEARTBEAT_INTERVAL": 30,
    "SCREENSHOT_INTERVAL": 15
}
```

Alternativamente, se pueden usar variables de entorno:

Variable	Valor por defecto	Descripción
`CLIENT_WS_URL`	`wss://localhost/ws/client/`	URL del WebSocket del servidor
`CLIENT_HEARTBEAT_INTERVAL`	30	Segundos entre heartbeats
`CLIENT_RECONNECT_DELAY`	5	Segundos antes de reintentar conexión
`CLIENT_SCREENSHOT_INTERVAL`	15	Segundos entre capturas de pantalla
`CLIENT_LOG_LEVEL`	INFO	Nivel de log

Cuadro 4: Variables de configuración del agente.

6.2. Instalación en Windows

1. Abrir una terminal (PowerShell o CMD) en la carpeta `client/`.
2. Ejecutar el instalador:

   ```
   python install_service.py install
   ```

3. El instalador realizará automáticamente:
   - Creación de un entorno virtual (`.venv`).
   - Instalación de las dependencias (`requirements.txt`).
   - Creación de un script VBS que oculta la ventana de consola.
   - Registro de una **Tarea Programada** en el Task Scheduler que arranca el agente al iniciar sesión.
   - Inicio inmediato del agente en segundo plano.

Comandos útiles en Windows:

```
# Ver estado
python install_service.py status

# Desinstalar
python install_service.py remove
```

```
# Consultar tarea programada
schtasks /query /tn ProyectoDAMClient
```

6.3. Instalación en Linux

1. Abrir una terminal en la carpeta `client/`.
2. Ejecutar el instalador con privilegios de superusuario:

   ```
   sudo python install_service.py install
   ```

3. El instalador realizará automáticamente:
   - Creación de un entorno virtual (`.venv`) e instalación de dependencias.
   - Generación de un fichero de unidad **systemd**.
   - Configuración de **polkit** para permitir el apagado remoto sin contraseña.
   - Habilitación e inicio del servicio.

Comandos útiles en Linux:

```
# Ver estado del servicio
sudo systemctl status proyectodamclient

# Ver logs en tiempo real
sudo journalctl -u proyectodamclient -f

# Reiniciar
sudo systemctl restart proyectodamclient

# Desinstalar
sudo python install_service.py remove
```

Importante: En Linux, el servidor gráfico debe ser **X11/Xorg** para que la captura de pantalla funcione. Se puede seleccionar en la pantalla de inicio de sesión (GDM, SDDM, LightDM).

6.4. Verificación

Una vez instalado, el equipo debería aparecer automáticamente en el **Dashboard** de la aplicación web como *en línea*. Si no aparece:

1. Verificar que la URL del WebSocket es correcta en la configuración del agente.
2. Comprobar que el servidor es accesible desde el equipo cliente (`ping tu-dominio.com`).
3. Revisar los logs del agente (`client_errors.log`).
4. Ejecutar una prueba rápida: `python install_service.py test`.

7. Configuración inicial de la aplicación

Tras el despliegue, acceder a la aplicación web y realizar la configuración inicial:

1. **Iniciar sesión** con las credenciales de superusuario definidas en el `.env`.
2. **Crear aulas:** Ir a la sección de Aulas y crear las aulas del centro con sus nombres.
3. **Registrar equipos de red:** Añadir los equipos MikroTik del centro con sus datos de conexión API.
4. **Crear grupos de usuarios:** Definir roles (ej.: *Profesor*) y asignar los permisos necesarios.
5. **Crear usuarios:** Registrar a los profesores, asignarles un grupo de permisos y las aulas que pueden gestionar.
6. **Configurar hosts permitidos:** Añadir las direcciones IP de los servidores que deben seguir siendo accesibles durante un bloqueo de internet (ej.: servidor de exámenes). Opcionalmente, asignar cada host a un aula específica.
7. **Instalar el agente** en los equipos de las aulas.

Los equipos cliente se registran automáticamente al conectarse por primera vez. Una vez aparezcan en el Dashboard, se pueden asignar a su aula correspondiente.

8. Comandos de administración

Comando	Descripción
`docker compose -f docker-compose.prod.yml ps`	Ver estado de los servicios
`docker compose -f docker-compose.prod.yml logs -f`	Ver logs en tiempo real
`docker compose -f docker-compose.prod.yml logs -f backend`	Logs solo del backend
`docker compose -f docker-compose.prod.yml exec backend sh`	Shell dentro del contenedor
`docker compose -f docker-compose.prod.yml down`	Detener todos los servicios
`docker compose -f docker-compose.prod.yml down -v`	Detener y borrar volúmenes (¡borra la BD!)
`docker compose -f docker-compose.prod.yml up -d --build backend`	Reconstruir solo el backend

Cuadro 5: Comandos de administración frecuentes.

9. Solución de problemas

Problema	**Solución**
No se encontró `.env`	Copiar `.env.example` como `.env` y rellenar los valores.
El backend no conecta con PostgreSQL	Verificar que `POSTGRES_HOST=db` y que el contenedor `db` esté *healthy* con `docker compose ps`.
El backend no alcanza el MikroTik	Comprobar que la red `backend_net` **no** tiene `internal: true`.
Frontend muestra página en blanco	Verificar que el build de Vite existe: `docker compose exec caddy ls /var/www/frontend/`.
El agente no se conecta al servidor	Revisar la URL del WebSocket en la configuración del agente. Comprobar conectividad con `ping`.
Captura de pantalla no funciona en Linux	Verificar que el servidor gráfico es X11/Xorg, no Wayland.
API de MikroTik rechaza la conexión TLS	Verificar que el certificado esté importado y asignado al servicio `api-ssl`. Comprobar que `api_port` sea 8729.

Cuadro 6: Problemas frecuentes y soluciones.

10. Estructura de ficheros del despliegue

Estructura del directorio produccion/

```
produccion/
  |-- .env.example              # Plantilla de variables de
     entorno
  |-- .env                      # Variables reales (no versionado
     )
  |-- docker-compose.prod.yml   # Orquestacion de servicios
  |-- Caddyfile                 # Configuracion del proxy inverso
  |-- deploy.sh                 # Script de despliegue (Linux/
     macOS)
  |-- deploy.bat                # Script de despliegue (Windows)
  |-- mikrotik_certs/           # Certificados extraidos de Caddy
  '-- README.md                 # Documentacion del despliegue
```

Estructura del directorio client/

```
client/
```

```
|-- client.py                  # Cliente principal (WebSocket)
|-- config.py                  # Configuracion (JSON + env vars)
|-- system_info.py             # Recopilacion de info del
   sistema
|-- install_service.py         # Instalador como servicio
|-- requirements.txt           # Dependencias Python
|-- client_config.json         # Configuracion local (opcional)
'-- README.md                  # Documentacion del cliente
```